Session 1: Intro

Created	@May 12, 2022 12:17 PM
Class	Reinforcement Learning
Type	
Materials	
Reviewed	☐
Property	

Learning objectives

- What is RL
- Should I use a given algorithim for this or not
- Should I use RL, supervised or unsupervised
- Dive into some of those algorithims

Course Schedule

Session	Type	Date	Start time	End time	Classrooms	Buildings	Topic
1	F2F	Apr 21st	12:00	13:20	MMB-603	MARIA DE MOLINA, 31 bis	Intro and admin
2	F2F	Apr 28th	12:00	13:20	MMB-603	MARIA DE MOLINA, 31 bis	RL101
3	F2F	May 3rd	12:00	13:20	MMB-603	MARIA DE MOLINA, 31 bis	Working with RL hyper-parameters
4	Activity	May 13th	N/A	N/A	N/A	N/A	Individual Challenge #1: "Houston we have a problem"
5	F2F	May 18th	11:00	12:20	MMB-602	MARIA DE MOLINA, 31 bis	Tabular methods I
6	F2F	May 19th	12:00	13:20	MMB-603	MARIA DE MOLINA, 31 bis	Tabular methods II
7	F2F	May 23rd	14:00	15:20	MMB-603	MARIA DE MOLINA, 31 bis	Approximate methods I
8	Activity	May 26th	N/A	N/A	N/A	N/A	Individual Challenge #2: "Driven by Reinforcement Learning"
9	F2F	June 2nd	14:30	15:50	MMB-602	MARIA DE MOLINA, 31 bis	Approximate methods II
10	F2F	June 9th	12:00	13:20	MMB-603	MARIA DE MOLINA, 31 bis	Policy Gradient
11	F2F	June 15th	14:00	15:20	MMB-603	MARIA DE MOLINA, 31 bis	Group Presentations I: SOTA for RL
12	F2F	June 16th	14:30	15:50	MMB-602	MARIA DE MOLINA, 31 bis	Group Presentations II: SOTA for RL
13	Activity	June 23rd	N/A	N/A	N/A	MARIA DE MOLINA, 31 bis	Practical Demo: "Can we get rich with RL?"
14	F2F	June 30th	14:30	15:50	MMB-602	MARIA DE MOLINA, 31 bis	Final remarks and Summary
15	F2F	July 6th	14:00	15:20	MMB-603	MARIA DE MOLINA, 31 bis	Final Report

- FIrst we see the backbone/history of RL (main methods of RL); then in the last 3 years, there are a lot of algorithis that are now the state of the art for RL (very young). The algorithims of the last 3/4 years are what is called state of the art reinforcement learning.
- 15 minute presentation and write short report.
- Practical Demo → asynchronous session to watch. Use case that is very demanded "how to get rich"→ playing in the stock market.

- Final evaluation which is a report.

Evaluation

CRITERIA	PERCENTAGE
Class participation (live sessions, activities, etc.)	20%
Individual activities (x2)	20%
Group assignment	40%
Final report	20%

Why is RL important anyway?

It has become **specially important in the last 5 years,** it is really when we started to see the point of using another method different from supervised or unsupervised learning. Sometimes, those methods are simply not possible or practicle → you solve it in a way that is not feasible in the real world. We needed something different and that's when RL came in.

- RL is not only games. It is applied in other industries.
- Important for modern data scientists
- RL provides mechanisms to deal with complex environments

OpenAI, Unity, Tesla, AWS, Microsoft Azure,and Google main pushers of RL

- OPenAI (founder was Elon Musk) is a company that since the beginning was thinking of doing research on ML and doing it open source
  - Main leaders of RL
- Unity is a gaming design company, tricky experienced company - metaverse.
  - investing a lot in RL.
- Tesla → how a Tesla car drives by itself when enabling the autodrive in the Tesla (complex combination of things but one of those is a complex RL algorithim)
- Then the cloud providers: AWS, Azure, Microsoft Azure and Google Cloud also have a lot to say in those areas.

When is RL needed when supervised and unsupervised learning not needed?

- Learning as it goes and adapting to the failure.
- Sometimes you don't have training data set.
- Rewards actions to achieve a goal.

We are not trying to predict an outcome, predict the probability of something happening like in supervised learning example. We are not even trying to classify/cluster some events data like we are doing in unsupervised. Instead:

> We are trying to teach a system to achieve **a goal.** We want a system to learn how to do something (that can take many forms and ways).

Examples

Multi-Agent Hide and Seek (Open AI)

Some years ago Open AI started to work on a system that was tring to learn it how to play hidden and seak.

- Virtual world with some agents that are acting like Police and other like Thiefs (some hide and some look).
- How to teach a system how to hide and discover techniques to hide and vice versa.

So how do agents acquire these skills?

They are trained using RL

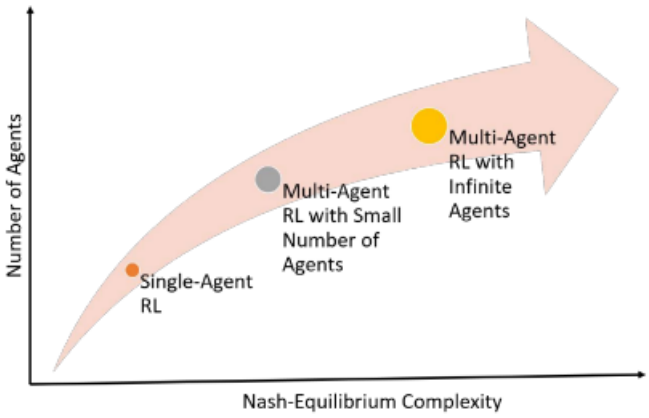
Reinforcement e¡learning is an algorithim inspired by the way animals learn. The agents play thousands of hidden and seek in parallel for many days. Train against each other, as well as past versions of themselves using an algorithim called self-play.

- Co-evolution and competition on earth lead to the only intelligent species today.
- Simple Rules lead to incredibly intelligent behaviour.
- Much larger and more diverse environment, truly complex agents will one day emerge.

RL for Maximizing Reward

We are talking about a system that is able ot create new ways of solving problems; that humans cannot solve e.g. creating shapes that even humans cannot find.

We are going to be learning different types of systems in this course (the msot complex type if the “Multi-Agent RL with Infinite Agents”. We are going to see “Single -Agent RL” with only one agent. As you increase the complecity, you need more power and everything becomes more complex.



Nash Equilibrium (Game Theory)

Movie beautiful mind → movie where there is a particular scene where this guy, creator of theory , in a bar they are tyring how to flirt with the girls, the blond gir was the most attractive. If all guys wanted with that one neither would be able to and the others would be offended therefore only go with the others. So that the team gets the most benefit, they should go with the others. This is what is called the Nash Equilibrium.

> The nash equilibrium is a theory that studies how to maximize a profit when you have multiple agents and variables in place.

- Real world example: the stock market and the finance works in this way. You have several actions you are playing; you win and loss in some of them but overall you have to maximize the profit.
- All of these theories are trying to achieve a goal which is to maximize a variable at the end → we call it **reward** (the formal RL way of defining that) → anything you want to maximise or minimize (e.g. minimize losses)
- Prisonner's dilemma is a specific theory

Example 2: Gaming Doda (Open AI)

How computer is able to come with better solutions than humans.

Open AI developed a model that was facing real professional players on a game called Doda. They trained a but that was going to play as a team and face all the other competitors/champions.

- The but started created strategies and started learning from what the humans were doing and that were even more creative than these champions and at the end they were beaten by them.
- It is amazing how it created strategies and *adapting in real time* (it is learning on the fly)
    - You dont train model and evaluate At the same time, we are *training, using it and running the inference in the same flow.*

Example 3: Alpha goal

- How do you take that, that is in virtual world? It is gaming, relatively easy as you have an environment that is already in the algorithim/in the system so you can probably adapt some algorithims to it but what these guys did is taking it into the real world.
- These guys said, we can also take it to the real world e.g. they trained this robotic hand to be able to manipulate objects → tried to solve it using a robotic cube. This model should be able to solve the cube by itself which mean moving the pieces with one hand.
- They found sme challenges like physics → the cube was sliping at the begining from the hand so they hand to put some gloves and change the hand. They were many iterations of the robotic hand.
- The goal was the actually mimic a human and combine that with the actual solution of the Rubbix cube; they then tried to mess around with the hand and put some objects. The hand was able to adjust and do it with the other fingers.

Example 4: Unity

- Company specialized in gaming. They tried to create a simple environment where robot was learning how to drive and park a car. This is real reinforcement learning (second challenge).
- At the beginning, you are going to see that is an awful model.
- RL requires a lot of iterations for being able to learn. In reality, those iterations are happening really fast.
- RL needs a lot of iterations (thpusands) to get closer to a solution to what a human considers a good solution.
    - It can always be improved because sometimes it gets a lot of movies to park and sometimes it even crashes so when do we STOP?

There is no way to measure errors. We do not have a dataset to compare, we cannot say it has 96% accuracy. So when do we decide to stop?

- Based on the goal? The goal can be binary? How do you make it more regular so that you know it is a goog solution to put it into production?
- Based on the errors if we don’t have something to compare?
- Compare with the frequency of cashes of the humans? It is not supervised learning (% of error)

> 💡 We are going to see trends, see how the model rewarding. The reward tells you how good is your model.

Eventually model is trying to explore new solutions and exploit what it is learning on the way. If this model is eventualkly doing a trend that is improving, it means that is still finding a good solution.

- We are going to calculate the **gradient** - if slope if still inclined to improving; it means that there is room for improvement i.e. error (compared to previous iterations).
- If we see model is stable or even getting worse → our error is now close to 0 (at the best solution pretty much). That is a **relative error** for deciding when to stop.

We no longer have an accuracy, we no longer have a perfect model. It is all the time learning; all the time trying to improve (all time possible to improve).

- Library where we have an environment where we can simulate physical areas/scenarios (gym for simulating the physics of the moon) so model is able to do things and measure the outocme for every action we take in the environment because in RL, you need an environment because you dont always have a real environment.
- In RL we need an enronment, we cant do it in a real environment sometimes so you sometimes neeed a virtual environment to then apply it in the real world.

Video: how a tesla car/autonomous car receives information.

- Not only using RL but a combination of things.
- A single camera is collecting all that data seeig in the screen (sensors, object detection, segmentation, supervised learning but one of them is an algorithim of RL because it has to decide on the fly on something it has never seen before
  - E.g. a women with green dress crossing the road a model **generalizes** to different situations).
  - How to see that? We can test the model in a different track in your computer. Model could crash or it could go well and **adapt to environment**.

A deep razor – you have an environment in the cloud where you simulate a track and you trian your model to discover a solution for doing the fastest lap in that circuit. Then, you adapt to a real car and run a full competition. This is a world wide competition that we can try.

The idea is going to be how to touch reinforcement learning in real world environment. This is going to happen in madrid IFEMA may 4th.

Basic Concepts

When is RL relevant? Supervised vs Reinforcement Learning

In supervised learning you have a dataset and you predict the labels (typical problem)

- Uses a dataset to estimate relationships between features and a label. These estimates are then used on unlabeled data in order to predict new labels

In RL we have **live** data. We do not have pretrained data. We do not have labelled data. We are collecting live data from the actions we are taking and we are using that to achieve a given goal.

- Uses live data to predict what strategy is best in order to achieve a goal. Where Supervised Learning begins with a complete dataset, RL collects its data as a part of the process.That happens in a process.

- E.g. we have to teach a robot to stand up. The robot has to discover the movements to stand up e.g. put hand then use the legs to stand up. How do we get to teach the robot to do this?

What are the problems in RL?

- We have to run a ton of iterations for collecting the data that you need for teaching the robot. That data should include all the possible ways the robot can move. That is millions of millions of data points that you have to collect. This means a lot of time and resources.
- Another problem is that in SL you are probably solving a regression/classification : what is the probability that robot is going to stand if robot does this or classify actions as positive or negative but we are **not achieving an outcome**.

Therefore problems area:

1. Requirement for a lot of data
2. We have a prediction but we do not have a goal

E.g. we are part of a campaign team for a precidential candidate in politics and it is asking us, what are the chances of winning → supervising model (20% probability) but now what should I do? In one situation, you are doing a prediction and in the other, a list of actions.

So how do we do that in RL? Solution

Defining states of an environment. A state is a given situation of our agen in our environment e.g. initial state= robot on floor and then I have a list of actions which are the possible movements of the robot e.g. bend left leg, right leg.

We are going to be testing each one of these actions where we assign a reward. To assign a reward we are going to use a policy. A policy is python code, we use that to do action and calculating reward. E.g. push left arm assign 0.3 and so on. Policy is used for doing the actions and calculating the reward.

- Assigning a value to each one of my actions. How do I do that? I will do that *based on my outcome* → tweak policy.
- When have full list of actions, you decide what is the best action (does not mean it is the best action). It means from the list of actions that you have “this is the best one so far”.
- You take that action and now you are in a *new state*.
- Eventually I will get to a situation in which i can define the action that is leading to the objecting which in this case is to making the robot stand up.

How is the learning happening?

You first define policy (piece of script) and then a list of hyperparameters and you pass that to your agent and your environment.

Based on those iterations you are going to be updating the policy. To update policy we can maybe use a quadratic equation. This is a mathematical function where you can input whatever you need.

I have this given use case, should I use this algorithim or not. Picking right algorithim, should I use this algorithim or not. Should I use deep learning or not?

Policy is normally predefined but sometimes not.